register itself to *lttng-sessiond* so that traces can be seamlessly written to the tracebuffer.

To make *thttpd* run, we will need a *config* file (which *thttpd* does not provide by default). Here is *config* from the official *thttpd* Fedora package:

```
# Save this as thttpd.conf
dir=/var/www/thttpd
chroot
user=thttpd         # default = nobody
logfile=/var/log/thttpd.log
pidfile=/var/run/thttpd.pid
```

I have prepared a version of *thttpd* for you that contains the above instrumentation as well as the *thttpd.conf*. You will need the *conf* file to start *thttpd* successfully in the next section. Get it from *http://step.polymtl.ca/~suchakra/osfy/thtpd-instrumented.tar.gz*

Start tracing

So now you are all set! Let's start our usual tracing routine. It's similar to kernel tracing.

```
$ lttng create osfy-thttpd
$ lttng enable-event -a -u        # enable all userspace
events
$ lttng start
$ sudo ./thttpd -C ./thttpd.conf      # start instrumented
thttpd
```

Now you can start loading your server-maybe by just opening a browser, pointing to *http://localhost* and simulating a load on the server by Apache Bench. I just tried connections from my browser for now.

```
$ lttng stop
$ lttng view

Trace directory: /home/suchakra/lttng-traces/osfy-
thttpd-20140120-172551

[17:26:24.474009892] (+?.?????????) isengard.localdomain
thttpd:count: { cpu_id = 2 }, { count = 1 }
[17:26:24.474044119] (+0.000034227) isengard.localdomain
thttpd:count: { cpu_id = 2 }, { count = 2 }
[17:26:38.936110649] (+14.462066530) isengard.localdomain
thttpd:count: { cpu_id = 3 }, { count = 3 }
[17:26:38.936246752] (+0.000136103) isengard.localdomain
thttpd:count: { cpu_id = 3 }, { count = 4 }
[17
```

You can observe that the five events were recorded and the connection count (our payload) is increasing. The delta time shows how much I waited between the requests and how many total requests I made while tracing. It's now time to finally end this tracing session.

```
$ lttng destroy
```

You can also record kernel as well as userspace traces at the same time for a more comprehensive view of the complete system with *thttpd*. Just enable the kernel and userspace events before starting a trace:

```
$ lttng enable-event -a -k
$ lttng enable-event -a -u
```

You can also add more information to the events or the trace channels by using *contexts*. For example, if you wish to add a Perf 'cache misses' counter value to all events recorded in the trace, you can use *add-context* before starting a trace:

```
$ lttng add context -k -t perf:cache-misses
```

There are many other cool things that you can do, such as filtering the traces, etc. Just browse through the man pages for *lttng* and *lttng-ust* to get more ideas.

Trace viewing and analysis

As I mentioned the last time, we can use various tools other than the command line based *babeltrace* to help us view and analyse Common Trace Format (CTF) traces. There are a couple of graphical tools available, but I will only cover the most comprehensive one-TMF. In order to maintain the continuity of our *thttpd* experiment, let's trace it again while enabling the userspace events (*-a -u* options, as mentioned above) in addition to kernel events. Run the trace for some time while making some HTTP requests and generate the traces. I have already generated some traces if you need those. Get them from *http://step.polymtl.ca/~suchakra/osfy/osfy-thttpd-20140127-194024.tar.gz*. We will need it later.

Setting up TMF

There are two ways to use TMF. One is the standalone Rich Client Platform (RCP) version. You can grab the latest Trace Viewer RCP from *http://secretaire.dorsal.polymtl.ca/~alexmont/tracing-rcp/* . Otherwise, if you are a regular Eclipse user and prefer to install TMF as an Eclipse plugin, download the latest version of the Eclipse IDE for C/C++ developers, which already includes Linux Tools and the LTTng plugins. On other versions, you can add the plugin using the '*Help→Install New Software...*' option. TMF is under the'Linux Tools' section of the main Eclipse repository.

Now you are all set and ready to roll! I will be using the Eclipse plugin to explain how to use TMF. The RCP version is no different as it is the plugin; it's just without the whole Eclipse framework.